LABORATORIO 7.2.: IDENTIDAD VISUAL CON MATERIAL YOU

1. Contexto y Objetivos de Aprendizaje

Contexto

En el desarrollo de aplicaciones modernas bajo la API 36, la interfaz de usuario debe trascender la estaticidad. Material You (Material Design 3) introduce el concepto de "Color Dinámico", permitiendo que la aplicación armonice con las preferencias estéticas del sistema operativo del usuario. En este laboratorio, se busca transformar la estética de la aplicación del examen del RDA-1 "SafePass 2026" en una herramienta accesible, moderna y altamente personalizada.

Objetivos de Aprendizaje

- Configurar el esquema de colores dinámicos (dynamicColorScheme) para adaptar la aplicación al entorno del usuario.
- Integrar fuentes personalizadas de Google Fonts mediante la configuración técnica de Type.kt.
- Implementar componentes de Material Design 3 (TopAppBar) que reaccionen a la jerarquía de colores del sistema.

2. Requisitos Previos

- Contar con el proyecto "SafePass 2026" funcional (procedente del Examen Práctico del RDA-1).
- Android Studio Ladybug 2024.2.1 instalado.
- Conexión a internet para la descarga de fuentes tipográficas bajo demanda.

3. Descripción General

Se llevará a cabo la personalización estética de la aplicación mediante la manipulación de los archivos de configuración del tema (Theme.kt y Type.kt). Se pasará de un diseño genérico a uno basado en Material You, asegurando que el contraste y la legibilidad se mantengan óptimos según los estándares de accesibilidad actuales.

4. Paso a Paso Detallado

Fase 1: Preparación de Dependencias y Certificados

Para el uso de fuentes externas sin incluir archivos binarios pesados, se requiere configurar el acceso a Google Fonts.

1. **Navegación:** En el panel *Project* de Android Studio, abrir la carpeta Gradle Scripts.
2. **Archivo:** Localizar y abrir build.gradle.kts (Module :app).
3. **Acción:** Dentro del bloque dependencies { ... }, agregar la siguiente librería al final: implementation("androidx.compose.ui:ui-text-google-fonts:1.7.0")

4. **Sincronización:** Hacer clic en el botón **"Sync Now"** en la parte superior derecha de la ventana.
5. **Verificación:** Navegar a app/src/main/res/values/. Si no existe el archivo font_certs.xml, se debe asegurar que el sistema tenga acceso a los certificados de Google Fonts para la descarga segura.

Fase 2: Definición de Tipografía Personalizada

Se procederá a configurar una fuente moderna (ej: "Poppins") para sustituir la fuente por defecto del sistema.

1. **Ruta del Archivo:** app/src/main/java/[tu.paquete]/ui/theme/Type.kt
2. **Acción:** Reemplazar el contenido del archivo por el siguiente bloque de código:

```
package com.example.safepass2026.ui.theme //asegurarse que el nombre de su paquete sea el correcto

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp
import androidx.compose.ui.text.googlefonts.GoogleFont
import androidx.compose.ui.text.googlefonts.Font
import com.example.safepass2026.R

// Configuración del proveedor de fuentes
val provider = GoogleFont.Provider(
    providerAuthority = "com.google.android.gms.fonts",
    providerPackage = "com.google.android.gms",
    certificates = R.array.com_google_android_gms_fonts_certs
)

val fontName = GoogleFont("Poppins")

val poppinsFontFamily = FontFamily(
    Font(googleFont = fontName, fontProvider = provider)
)

// Aplicación de la fuente a la jerarquía de Material3
val Typography = Typography(
    titleLarge = TextStyle(
        fontFamily = poppinsFontFamily,
        fontWeight = FontWeight.Bold,
        fontSize = 22.sp
    ),
    bodyLarge = TextStyle(
```

```
        fontFamily = poppinsFontFamily,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp
    )
)
```

- **Nota:** El uso de GoogleFont.Provider permite que la aplicación sea más ligera, ya que la fuente es gestionada y cacheada por los servicios de Google Play.

Fase 3: Activación del Color Dinámico

Se debe asegurar que la lógica del tema reconozca si el dispositivo soporta Material You (Android 12+ / API 31+).

1. **Ruta del Archivo:** app/src/main/java/[tu.paquete]/ui/theme/Theme.kt
2. **Acción:** Localizar la función SafePass2026Theme y verificar que la variable dynamicColor se procese correctamente:

```
@Composable
fun SafePass2026Theme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true, // Debe estar en true para soportar Material You
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
    // ... resto del código del MaterialTheme
}
```

Fase 4: Implementación de la Interfaz con TopAppBar

Se creará una pantalla de configuración para visualizar los cambios aplicados.

1. **Ruta del Archivo:** app/src/main/java/[tu.paquete]/MainActivity.kt
2. **Acción:** Agregar el siguiente Composable de prueba:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ConfigScreen() {
    Scaffold(
```

```
      topBar = {
        TopAppBar(
          title = { Text("SafePass 2026 - Identidad", style = MaterialTheme.typography.titleLarge)
},
          colors = TopAppBarDefaults.topAppBarColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer,
            titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer
          )
        )
      }
    ) { innerPadding ->
      Column(modifier = Modifier.padding(innerPadding).padding(16.dp)) {
        Text(
          text = "Bienvenido al futuro de la seguridad móvil.",
          style = MaterialTheme.typography.bodyLarge
        )
      }
    }
}
```

5. Resultados Esperados

Al ejecutar la aplicación en el emulador con API 36:

1. La tipografía debe mostrarse con el estilo de la fuente "Poppins" (más redondeada y moderna).
2. Al cambiar el fondo de pantalla del emulador (Wallpaper), el color del TopAppBar y de los textos principales debe cambiar automáticamente para combinar con los colores predominantes del fondo elegido.

6. Desafío Extra

Tarea: Implementar un componente FloatingActionButton (FAB) que utilice el esquema de color secundario del sistema (secondaryContainer).

Resolución:

Agregar dentro del Scaffold en la MainActivity.kt:

```
floatingActionButton = {
  FloatingActionButton(
    onClick = { /* Acción */ },
    containerColor = MaterialTheme.colorScheme.secondaryContainer
  ) {
    Icon(Icons.Default.Add, contentDescription = "Agregar")
  }
```

}

7. Entregables

1. Informe **PDF** con:
    - Código MainActivity.kt, Theme.kt y Type.kt modificados.
    - Captura de pantalla del emulador con dos fondos de pantalla distintos, demostrando el cambio de color dinámico en la App.
    - Explicación de cómo influye el colorScheme en la accesibilidad de la aplicación.
    - Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.

8. Criterios de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe